

Essay on Solving Intersection and Area Problems Using Python

Introduction

In this report, we will see how programmatically we solve the mathematical problem of finding the intersection points and the areas under and between a curve and a line. The primary objectives were to determine the intersection of the curve ($y = 5 + 4x - x^2$) and the line ($y = 10 - 2x$), compute the area under both the curve and the line, and finally find the area between them. This problem was addressed using Python libraries such as NumPy, SymPy, and Matplotlib for numerical operations, symbolic mathematics, and visualization, respectively.

Code Explanation

The main function “graph()” takes two lambda functions representing the equations and a range of x-values as inputs. This function visualizes the equations on the same plot, finds their intersection points, and calculates the areas under the curve and the line.

1. Finding Intersection Points

The intersection points are determined by equating the two equations and solving for x. This was accomplished using SymPy's symbolic equation solving capabilities:

```
# Define the variable
x = sp.symbols('x')

# Find the points of intersection by setting y1 equal to y2
intersection_eq = sp.Eq(formula1(x), formula2(x))

# Solve the equation for x
intersection_points x = sp.solve(intersection_eq, x)
```

The corresponding y-values are then calculated using one of the equations.

2. Calculating Area Under the Curve

The area under the curve ($y = 5 + 4x - x^2$) was calculated using the Trapezoidal Rule. This is implemented in the function “area_curve()”, which takes the x-values of the intersections and evaluates the integral numerically:

```
def area_curve(p1, p2, formula):
    """ Calculate the area under the curve using the Trapezoidal Rule. """
    x1, y1 = p1
    x2, y2 = p2

    # Number of trapezoids
    num_trapezoids = 1000
    x_values = np.linspace(x1, x2, num_trapezoids + 1)
    y_values = formula(x_values)

    area = np.trapz(y_values, x_values)
    return area
```

3. Calculating Area Under the Line

The area under the line ($y = 10 - 2x$) was calculated using the Trapezoidal Rule. This is implemented in the function “area_line()”, which takes the x-values of the intersections and evaluates the integral numerically:

```
def area_line(p1, p2, formula):
    """ Calculate the area under the line using the Trapezoidal Rule. """
    x1, y1 = p1
    x2, y2 = p2

    # Number of trapezoids
    num_trapezoids = 1000
    x_values = np.linspace(x1, x2, num_trapezoids + 1)
    y_values = formula(x_values)

    area = np.trapz(y_values, x_values)
    return area
```

4. Finding Area Between the Curve and the Line

The final step involves subtracting the area under the line from the area under the curve, thereby yielding the area between the two. This was efficiently handled by the code using a simple subtraction operation:

```
# Calculate the area between the curve and the line
area_between_curve_and_line = round(area_curve_value - area_line_value, 3)
print(f"\nArea between the curve and the line: {area_between_curve_and_line:.3f}")
```

Alternative Approach

An alternative approach could involve using analytical methods to solve the integrals for areas under the curve and line or we can use different python libraries like Scipy numerical integration method “`scipy.integrate.quad`” or we can use Riemann Sums/Rectangle Method, where we need to provide manual implementation without any python libraries.

For plotting we can use different python libraries like seaborn, plotly and more.

References

Asutkar, S. (n.d.). *A Step-by-Step Guide to the Trapezoidal Rule*. Retrieved from askpython.com:

<https://www.askpython.com/python-modules/numpy/numpy-trapz>

Learning, G. (n.d.). *SIG742 - Modern Data Science*. Retrieved from olympus.mygreatlearning.com:

<https://olympus.mygreatlearning.com/courses/103477>

Sharma, R. (n.d.). *Introduction to SymPy for Symbolic Mathematics*. Retrieved from medium.com:

<https://medium.com/pythoneers/introduction-to-sympy-for-symbolic-mathematics-d03788f2fc15>

Conclusion

The code implemented successfully addresses the problem of finding intersections and areas related to a curve and a line. The numerical approach adopted using Python provides a flexible and efficient solution, easily adaptable for different functions and ranges. Understanding these fundamental mathematical operations and their implementation in programming is essential for tackling various applied mathematics and engineering problems. Future work could explore using different numerical methods or extending the solution to handle 3D surfaces and their intersections.